

Half Adder

Dataflow

```
1  module half_adder_df (  
2      input logic a, b,  
3      output logic sum, carry  
4  );  
5      assign sum = a ^ b;  
6      assign carry = a & b;  
7  endmodule
```

Behavioral

```
Behavioral > ≡ halfAdderBH.v  
1  module half_adder_bh (  
2      input logic a, b,  
3      output logic sum, carry  
4  );  
5      always_comb begin  
6          sum = a ^ b;  
7          carry = a & b;  
8      end  
9  endmodule
```

Structural

```
Structural > ≡ halfAdderST.v  
1  module half_adder_st (  
2      input logic a, b,  
3      output logic sum, carry  
4  );  
5      // Using built-in gate primitives (positional only)  
6      xor sum_gate (sum, a, b);  
7      and carry_gate (carry, a, b);  
8  endmodule  
9
```

Full Adder Dataflow

```
1  module full_adder_df (  
2      input logic a, b, c,  
3      output logic sum, carry  
4  );  
5      assign sum = (a ^ b) ^ c;  
6      assign carry = (a & b) ^ ((a ^ b) & c);  
7  endmodule
```

Behavioral

```
1  module full_adder_bh (  
2      input logic a, b, c,  
3      output logic sum, carry  
4  );  
5      always_comb begin  
6          sum = (a ^ b) ^ c;  
7          carry = (a & b) ^ ((a ^ b) & c);  
8      end  
9  endmodule
```

Structural

```
1  module full_adder_st (  
2      input logic a, b, c  
3      output logic sum, carry  
4  );  
5      wire temp1, temp2, temp3;  
6      // Using gate primitives  
7      xor x1(temp1, a, b);  
8      xor x2(sum, temp1, c);  
9  
10     and a1(temp2, a, b);  
11     and a2(temp3, temp1, c);  
12     xor x3(carry, temp3, temp2);  
13 endmodule
```

Pre Lab

1. What distinguishes combinational circuits from sequential circuits?

- Combinatorial circuits depend on the current input values to produce outputs, sequential circuits depend on both the current inputs and the previous state stored in memory elements to produce an output.

2. Derive the sum and carry expressions for half and full adders from their truth tables.

- Half Adder

A	B	$(A \oplus B)$ (Sum)	$(A \& B)$ (Carry)
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

- Full Adder

A	B	C	$A \oplus B$	$(A \oplus B) \oplus C$ (Sum)	$(A \oplus B) \& C$	$A \& B$	$(A \& B) \& ((A \oplus B) \oplus C)$ (Carry)
0	0	0	0	0	0	0	0
0	0	1	0	1	0	0	0
0	1	0	1	1	1	0	0
0	1	1	1	0	0	0	1
1	0	0	1	1	1	0	0
1	0	1	1	0	0	0	1
1	1	0	0	0	0	1	1
1	1	1	0	1	0	1	1

3. Derive the difference and borrow expressions for half and full subtractors.

- Half Subtractor

A	B	$\sim A$	$A \oplus B$ (Difference)	$\sim A \& B$ (Borrow)
0	0	1	0	0
0	1	1	1	1
1	0	0	1	0
1	1	0	0	0

- Full Subtractor

A	B	C	$A \oplus B$	$A \oplus B \oplus C$ (Difference)	$\sim A$	$\sim A \& B$	$\sim(A \oplus B)$	$\sim(A \oplus B) \& C$	$\sim(A \oplus B) \& C$ $\parallel \sim A \& B$ (Borrow)
0	0	0	0	0	1	0	1	0	0
0	0	1	0	1	1	0	1	1	1
0	1	0	1	1	1	1	0	0	1
0	1	1	1	0	1	1	0	0	1
1	0	0	1	1	0	0	0	0	0
1	0	1	1	0	0	0	0	0	0
1	1	0	0	0	0	0	1	0	0
1	1	1	0	1	0	0	1	1	1

4. Explain the advantages of SystemVerilog's logic data type over traditional Verilog's reg and wire.

- SystemVerilog's logic simplifies code by merging the capabilities of wire and reg, reduces errors during refactoring, and improves readability—making it the preferred default for most single-driver signals in modern RTL design.

5. What is the purpose of the -g2012 flag in Icarus Verilog?
 - The -g2012 flag tells the compiler to enable IEEE 1800-2012 (SystemVerilog 2012) features.

6. What is the purpose of the \$dumpvars system task in Verilog testbenches?
 - The \$dumpvars system task in Verilog is used in testbenches to record signal activity into a Value Change Dump file for waveform viewing with tools like GTKWave.

7. What are the benefits of using named port connections (.port(name)) over positional connections?
 - Named port connections in Verilog (.port(signal)) are safer and easier to maintain than positional connections because they explicitly label each signal, improving readability and reducing errors. Unlike positional connections, they are unaffected by changes in port order, allow partial connections, and make refactoring or debugging simpler—especially in large or collaborative projects. While positional connections may be acceptable for very small, simple modules, named connections are considered best practice for most real-world designs.

Half Subtractor Data Flow

```
1 module half_subtractor_df (  
2     input logic a, b,  
3     output logic difference, borrow  
4 );  
5     assign difference = a ^ b;  
6     assign borrow = ~a & b;  
7 endmodule
```

Behavioral

```
1 module half_subtractor_bh (  
2     input logic a, b,  
3     output logic difference, borrow  
4 );  
5     always_comb begin  
6         difference = a ^ b;  
7         borrow = ~a & b;  
8     end  
9 endmodule
```

Structural

```
module half_subtractor_st (  
    input logic a, b,  
    output logic difference, borrow  
);  
    xor difference_gate (difference, a, b);  
    and borrow_gate (borrow, ~a, b);  
endmodule
```

Full Subtractor Dataflow

```
1  module full_subtractor_df (  
2      input logic a, b, c,  
3      output logic difference, borrow  
4  );  
5      assign difference = (a ^ b) ^ c;  
6      assign borrow = (~(a ^ b) & c) | (~a & b);  
7  endmodule
```

Behavioral

```
1  module full_subtractor_bh (  
2      input logic a, b, c,  
3      output logic difference, borrow  
4  );  
5      always_comb begin  
6          difference = (a ^ b) ^ c;  
7          borrow = (~(a ^ b) & c) | (~a & b);  
8      end  
9  endmodule
```

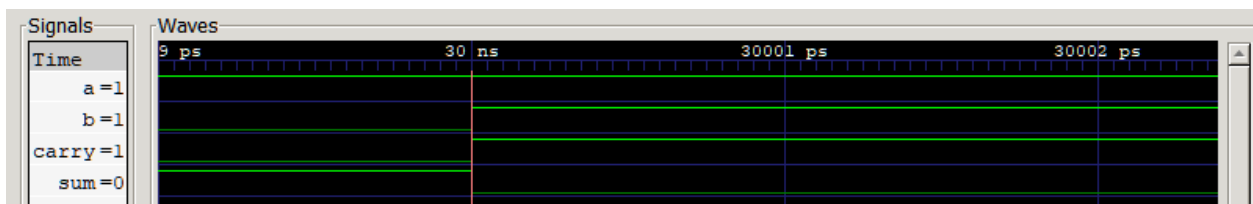
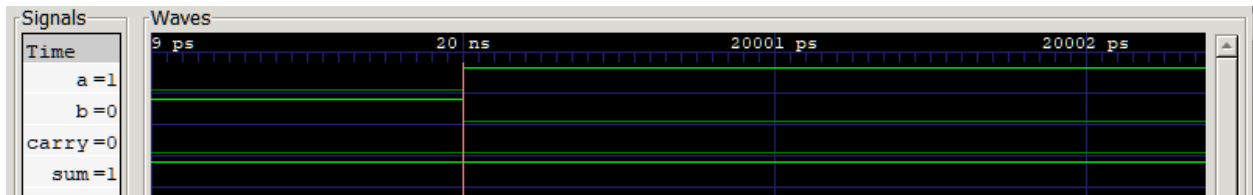
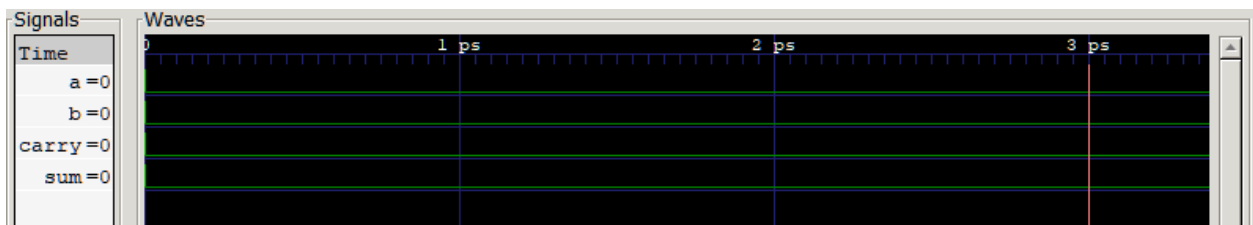
Structural

```
1  module full_subtractor_st (  
2      input logic a, b, c,  
3      output logic difference, borrow  
4  );  
5      wire temp1, temp2, temp3;  
6      // Using gate primitives  
7      xor x1(temp1, a, b);  
8      xor x2(difference, temp1, c);  
9  
10     and a1(temp2, ~a, b);  
11     and a2(temp3, ~temp1, c);  
12     or o3(borrow, temp3, temp2);  
13 endmodule
```

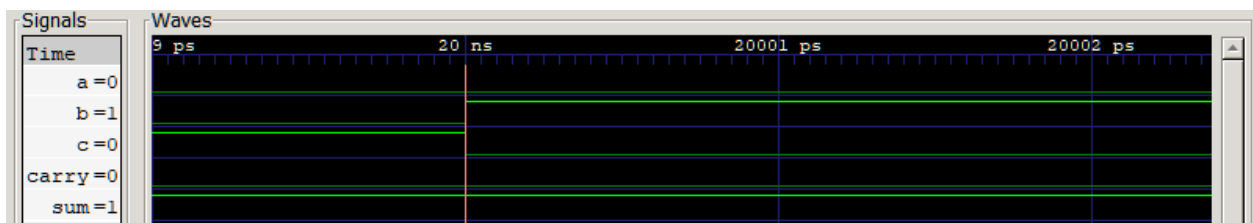
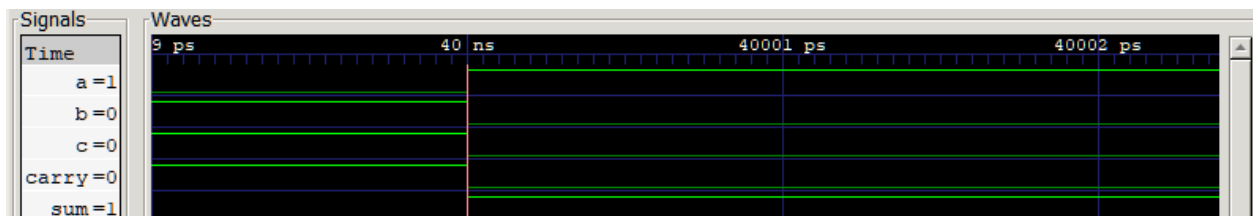
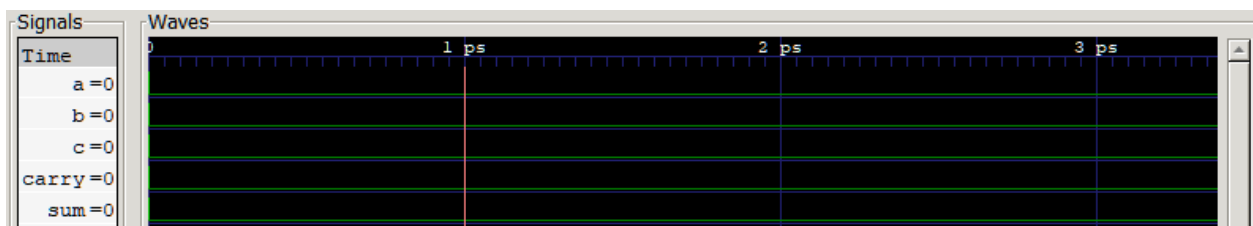
Post-lab Questions

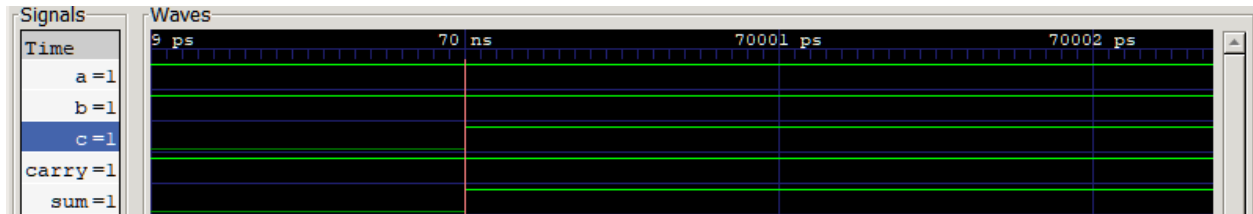
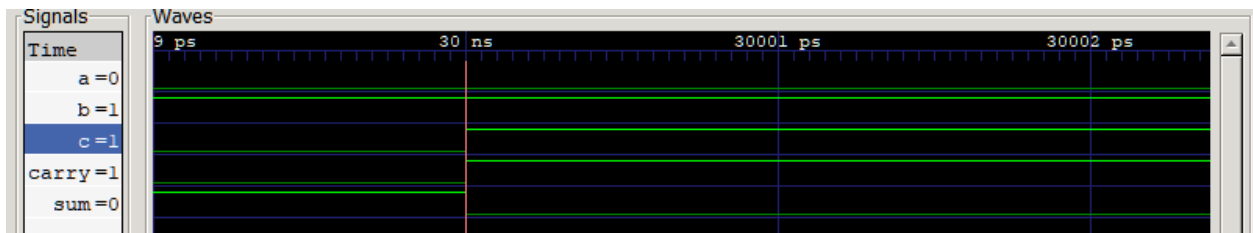
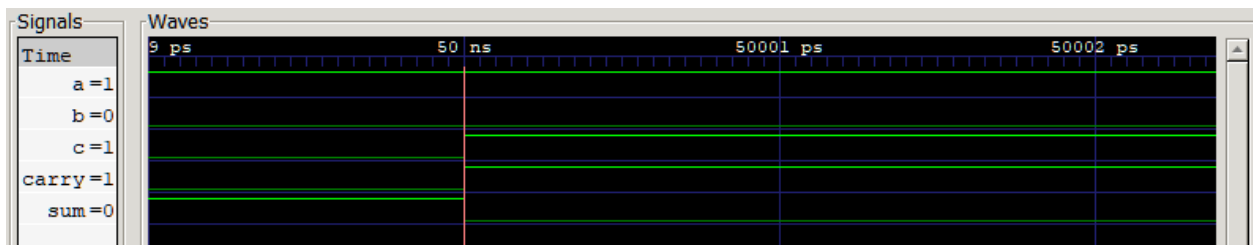
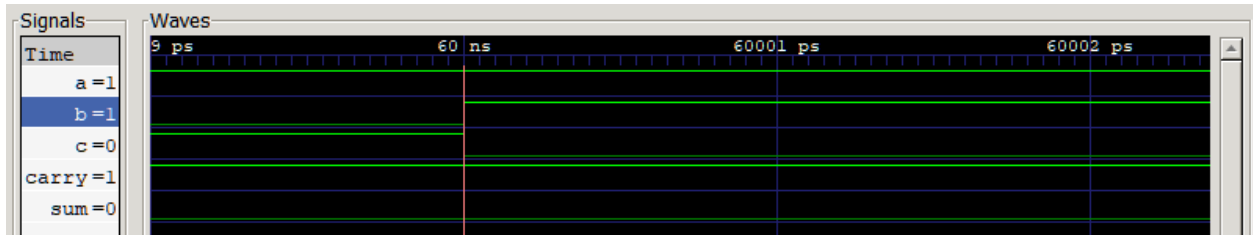
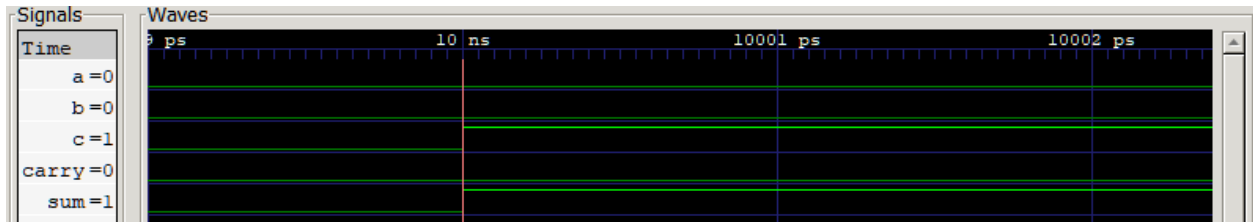
1. Compare and contrast the three SystemVerilog modeling styles used in this lab.
 - Behavioral modeling describes what a circuit does using high-level constructs like always blocks, making it easy to write but less directly tied to hardware. Dataflow modeling uses continuous assignments (assign) to show how data moves between signals, ideal for combinational logic. Structural modeling connects modules or gates to represent actual hardware, offering accuracy but being more verbose and harder to modify.
2. Explain the difference between continuous assignments and procedural assignments.
 - Continuous assignments (assign) update outputs automatically when inputs change and are used for simple combinational logic on wire types. Procedural assignments occur in always or initial blocks, executing only when triggered, and are suited for sequential or complex behaviors using reg or logic.
3. Describe the function of process statements and block statements in SystemVerilog.
 - Process statements (always, always_ff, initial) define hardware behavior triggered by events. Block statements group code: sequential blocks (begin...end) run statements in order, while parallel blocks (fork...join) run them simultaneously.
4. What are sequential statements, and how do they differ from concurrent statements?
 - Sequential statements execute step-by-step inside procedural blocks, like software instructions. Concurrent statements run in parallel, as with multiple assign statements, reflecting real hardware operating simultaneously.
5. Explain the difference between high-impedance (Z) and unknown (X) signal states.
 - A Z state means a signal is disconnected or not driven, common for shared buses or tri-state buffers. An X state means the value is unknown, caused by uninitialized variables or conflicting drivers, and helps detect design issues in simulation.
6. How does the adder-subtractor circuit implement subtraction using addition?
 - Subtraction is done by adding the two's complement of B to A. The circuit inverts B, adds one, and feeds this to an adder, using a control signal to switch between addition and subtraction.
7. What are the advantages of using always_comb instead of always @(*)?
 - always_comb automatically creates its sensitivity list, ensures outputs are fully driven, and prevents multiple drivers, making it safer and less error-prone than always @(*) for combinational logic.
8. How does SystemVerilog improve upon traditional Verilog for digital design?
 - SystemVerilog adds strong typing (logic, enum), specialized procedural blocks (always_ff, always_comb), interfaces, assertions, and object-oriented features, making designs clearer, safer, and easier to verify compared to Verilog.
9. Compare the readability and maintainability of traditional Verilog vs. SystemVerilog code.
 - SystemVerilog is more readable and maintainable than Verilog due to clearer syntax, stronger typing, and built-in verification features. Verilog is simpler but can become error-prone and harder to manage in large designs.

Half Adder

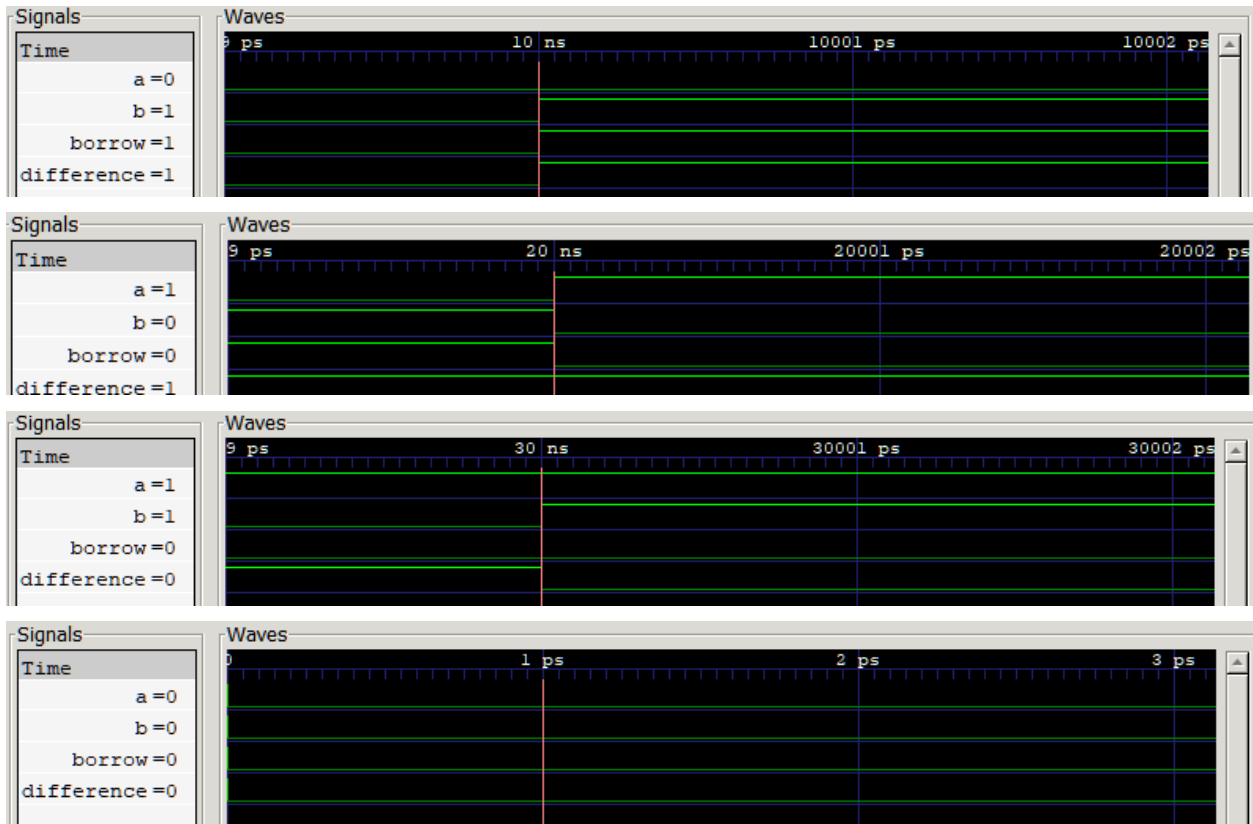


Full Adder





Half Subtractor



Full Subtractor

